



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ
ΤΟΜΕΑΣ ΤΕΧΝΟΛΟΓΙΑΣ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΩΝ
ΕΡΓΑΣΤΗΡΙΟ ΥΠΟΛΟΓΙΣΤΙΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

Εργαστήριο Λειτουργικών Συστημάτων
7ο Εξάμηνο, Ακαδημαϊκή περίοδος 2025–2026

Εργαστηριακή Άσκηση 3

Συστήματα Αρχείων σε Περιβάλλον Linux

oslab11

Αϊφαντή Ελισάβετ (03122821)

Βασίλειος Μιλτιάδης Πάντζος (03121281)

1ο Μέρος: Το σύστημα αρχείων ext2

fdisk1.img

1) Τροποποιούμε το [./utopia.sh](#) με τον εξής τρόπο:

```
# Over TCP to unix:csab.ece.ntua.gr:49152.
exec $QEMU -enable-kvm -M pc -m $UTOPIA_MEMORY_MB \
-smp $UTOPIA_VCPUS -drive file=$QCOW2_PRIVATE_FILE,if=virtio \
-drive file=./fsdisk1-7c2b425a6.img,format=raw,if=virtio \
-drive file=./fsdisk2-a0173283d.img,format=raw,if=virtio \
-drive file=./fsdisk3-982902777.img,format=raw,if=virtio \
```

Κάνοντας lsblk μέσα στο vm βλέπουμε το εξής:

```
root@utopia:~# lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sr0          11:0    1 1024M  0 rom
vda          253:0    0   10G  0 disk
└─vda1       253:1    0   10G  0 part /
vdb          253:16   0    50M  0 disk
vdc          253:32   0    20M  0 disk
vdd          253:48   0    20M  0 disk
```

Αρα βλέπουμε ότι το vm βλέπει ως δίσκους τις εικόνες μας (vdb,vdc,vdd).

Τώρα για την εικόνα fdisk1.img θα κάνουμε mount το vdb με τον εξής τρόπο:

```
root@utopia:/home/user/shared/ext2/ext2-vdisks# mkdir -p /mnt/fs1
root@utopia:/home/user/shared/ext2/ext2-vdisks# mount /dev/vdb /mnt/fs1
root@utopia:/home/user/shared/ext2/ext2-vdisks# ls -l /mnt/fs1
total 14
drwxr-xr-x 2 root root 1024 Dec 12 2023 dir1
drwxr-xr-x 2 root root 1024 Dec 12 2023 dir2
drwx----- 2 root root 12288 Dec 12 2023 lost+found
```

Πραγματοποιήθηκε **προσάρτηση (mount)** του συστήματος αρχείων τύπου ext2 που περιέχεται στον δίσκο της εικόνας fdisk1.img. Το σύστημα αρχείων ενσωματώθηκε στην ιεραρχία καταλόγων του Linux, συγκεκριμένα στο σημείο προσάρτησης (mount point) /mnt/utopia/fs1.

2) Στη στήλη 1K-blocks παρακάτω βλέπουμε τον αριθμό 49568. Αυτό σημαίνει 49.568 KB *ωφέλιμου* χώρου. Στο σύστημα αρχείων ext2, 5% της μνήμης του δίσκου ανατίθεται στον superuser, ενώ κάποιος χώρος χρησιμοποιείται για μεταδεδομένα. Αυτό αποτρέπει την μνήμη από το να γεμίσει εντελώς, το οποίο αποτρέπει τον κατακερματισμό. Συνεπώς, η συνολική μνήμη που καταλαμβάνει ο δίσκος θα είναι ελαφρώς μεγαλύτερη.

```
root@utopia:/home/user/shared/ext2/ext2-vdisks# df -l /mnt/fs1
Filesystem      Type 1K-blocks  Used Available Use% Mounted on
/dev/vdb         ext2   49568      16    46992    1% /mnt/fs1
```

Με sudo hexedit /dev/vdb βλέπουμε:

```
-- vdb --0x0/0x3200000--0%-----
```

Μας δίνεται το μέγεθος του δίσκου σε hex. Μετατρέποντας σε δεκαδικό, έχουμε ότι περιέχει 52,428,800 bytes, δηλαδή έχει μέγεθος **50MB**.

3) Περιέχει το σύστημα αρχείων ext2, όπως φαίνεται παραπάνω. Επισημάνθηκε με το -T flag.

Με τη χρήση hexedit, η υπογραφή (s_magic) του συστήματος αρχείων θα μας υποδείξει για τι σύστημα αρχείων πρόκειται. Στο πρότυπο ext2, αυτή η υπογραφή βρίσκεται πάντα στο offset 0x38 μέσα στο Superblock. Επειδή το Superblock έχει offset 1024 bytes (0x400), η θέση που ψάχνουμε είναι: $0x400 + 0x38 = 0x438$.

Στο Superblock, στο offset 0x438 (που είναι το πεδίο s_magic), εντοπίστηκαν τα bytes 53 EF. Λόγω της διάταξης Little Endian, η τιμή διαβάζεται ως 0xEF53, η οποία αποτελεί τη μοναδική υπογραφή (Magic Number) για τα συστήματα αρχείων της οικογένειας ext2/3/4.

4-7) Για τα ερωτήματα 4-7 χρησιμοποιήσαμε την εντολή dumpe2fs, η οποία παρέχει πληροφορίες σχετικά με το superblock και τα block groups του συστήματος αρχείων. Με grep, τυπώνουμε τις πληροφορίες σχετικά με το πότε δημιουργήθηκε το σύστημα αρχείων, πότε προσαρτήθηκε τελευταία φορά, σε ποιο μονοπάτι προσαρτήθηκε τελευταία φορά και πότε τροποποιήθηκε τελευταία φορά.

```
root@utopia:/home/user/shared/ext2/ext2-vdisks# dumpe2fs -h /dev/vdb | grep -E "created:|Last mount|Last write"
dumpe2fs 1.47.0 (5-Feb-2023)
Last mounted on:      /cslab-bunker
Filesystem created:   Tue Dec 12 15:23:16 2023
Last mount time:      Sun Jan 18 13:24:52 2026
Last write time:      Sun Jan 18 13:24:52 2026
```

Με την εντολή hexedit, βρήκαμε τις παραπάνω τιμές ως εξής:

4) Ψάχνουμε το πεδίο `s_mkfs_time`, στο offset 0x2C του superblock με μήκος 4 bytes.

```
0000042C  E4 7A 78 65
```

Σε little endian είναι 0x65787AE4. Αυτός είναι ο κατά σύμβαση ο αριθμός των δευτερολέπτων που έχουν περάσει από την 1η Ιανουαρίου 1970 (POSIX time). Αντιστοιχεί στην ημερομηνία: Tue Dec 12 15:23:16 UTC 2023 (περίπου, ανάλογα με τη ζώνη ώρας).

5) Ψάχνουμε το πεδίο `s_mtime` (Last mount time) στο offset 0x20 του superblock.

```
00000420  00 20 00 00
```

Σε little endian είναι 0x2000. Παρατηρούμε ότι δεν έχει γίνει update του πεδίου αυτού στο Primary superblock, μπορεί όμως να υπάρχει σε κάποιο backup superblock.

6) Στο offset 0x488 (πεδίο `s_last_mounted`), διαβάστηκε η ακολουθία ASCII χαρακτήρων που σχηματίζει το string `/cslab-bunker`.

7) Στο offset 0x430 (πεδίο `s_wtime`), βρέθηκαν τα bytes 83 E5 47 69 (little endian 0x6947E583). Σε δεκαδικό μας δίνει το Unix Timestamp: 1766352259, που αντιστοιχεί στην ημερομηνία Sun Dec 21 21:24:19 UTC 2025 (περίπου, ανάλογα με τη ζώνη ώρας).

8) Το μπλοκ (block) είναι η μικρότερη λογική μονάδα αποθήκευσης και διαχείρισης δεδομένων που αναγνωρίζει το σύστημα αρχείων, αποτελώντας μια αφαίρεση πάνω από τους φυσικούς τομείς (sectors) του δίσκου. Έχει σταθερό καθορισμένο μέγεθος (π.χ. 1KB ή 4KB) που ορίζεται κατά τη δημιουργία του συστήματος αρχείων, ενώ κάθε αρχείο δεσμεύει χώρο στο δίσκο αποκλειστικά σε ακέραια πολλαπλάσια αυτών των μπλοκ, ανεξάρτητα από το αν γεμίζει πλήρως τον χώρο τους ή όχι.

9) Βλέπουμε τι μέγεθος block χρησιμοποιεί το παραπάνω σύστημα αρχείων, ψάχνοντας για την παράμετρο Block Size με την `dumpe2fs`.

```
root@utopia:/home/user/shared/ext2/ext2-vdisks# dumpe2fs -h /dev/vdb | grep -E "Block size"
dumpe2fs 1.47.0 (5-Feb-2023)
Block size: 1024
```

Το πεδίο `s_log_block_size` έχει offset 0x18 από το superblock. Με hexedit:

```
00000414  01 00 00 00 00 00 00 00
```

Σύμφωνα με τον τρόπο λειτουργίας του ext2, αυτό σημαίνει ότι το μέγεθος υπολογίζεται ως $1024 * 2^0 = 1024$ bytes.

10) Το inode (index node) είναι μια θεμελιώδης δομή δεδομένων στα συστήματα αρχείων τύπου Unix (όπως το ext2). Κάθε αρχείο ή κατάλογος αντιστοιχίζεται σε ένα μοναδικό inode, το οποίο αποθηκεύει όλα τα μεταδεδομένα (metadata) του αρχείου, εκτός από το όνομα του αρχείου και τα ίδια τα δεδομένα του.

Συγκεκριμένα, το inode περιέχει:

- Τον τύπο του αρχείου και τα δικαιώματα πρόσβασης.
- Τον ιδιοκτήτη (UID) και την ομάδα (GID).
- Το μέγεθος του αρχείου.
- Τις χρονοσφραγίδες (δημιουργία, πρόσβαση, τροποποίηση).
- Τους δείκτες (pointers) προς τα μπλοκ δεδομένων (data blocks) όπου βρίσκεται αποθηκευμένο το περιεχόμενο του αρχείου στον δίσκο.

11) Βρήκαμε το μέγεθος του inode, ψάχνοντας για την παράμετρο Inode size με την dumpe2fs.

```
root@utopia:/home/user/shared/ext2/ext2-vdisks# dumpe2fs -h /dev/vdb | grep -E "Inode size"
dumpe2fs 1.47.0 (5-Feb-2023)
Inode size:                128
```

Στο offset 0x458 (πεδίο *s_inode_size*), βρέθηκαν τα bytes 80 00 (little endian 0x0080) δηλαδή 128 σε δεκαδικό.

```
000000450  00 00 00 00 0B 00 00 00 80 00 00 00
```

Άρα το κάθε inode καταλαμβάνει 128 bytes στο δίσκο.

12) Ομοίως, ψάχνοντας τις παραμέτρους Free blocks και Free inodes:

```
root@utopia:/home/user/shared/ext2/ext2-vdisks# dumpe2fs -h /dev/vdb | grep -E "Free blocks|Free inodes"
dumpe2fs 1.47.0 (5-Feb-2023)
Free blocks:                49552
Free inodes:                 12810
```

Στο offset 0x40C (*s_free_blocks_count*) βλέπουμε τα bytes 90 C1 00 00. Σε Little Endian αυτό είναι 0xC190.

```
000000408  00 0A 00 00 90 C1 00 00 0A 32 00 00
```

Άρα τα διαθέσιμα blocks είναι συνολικά 49.552.

Στο offset 0x410 (*s_free_inodes_count*) βλέπουμε τα bytes 0A 32 00 00. Σε Little Endian αυτό είναι 0x320A. Άρα τα διαθέσιμα inodes είναι 12.810.

13) Το Superblock είναι η πιο κρίσιμη δομή δεδομένων (metadata) σε ένα σύστημα αρχείων ext2. Λειτουργεί ως η "ταυτότητα" ολόκληρου του συστήματος αρχείων, καθώς περιέχει όλες τις απαραίτητες πληροφορίες για τη συνολική δομή και διαχείριση του δίσκου. Χωρίς αυτό, το λειτουργικό σύστημα δεν μπορεί να προσαρτήσει (mount) τον δίσκο.

Περιλαμβάνει πληροφορίες όπως:

- Το συνολικό αριθμό των μπλοκ και των inodes.
- Το μέγεθος του μπλοκ (block size).
- Τον αριθμό των ελεύθερων μπλοκ και inodes.
- Τον "Μαγικό Αριθμό" (Magic Number) για αναγνώριση του τύπου (ext2).
- Καταστάσεις σφάλματος και μετρητές προσάρτησης (mount counts).

14) Το πρωτεύον Superblock βρίσκεται πάντα σε μια σταθερή θέση στην αρχή του δίσκου: ξεκινάει στο offset 1024 bytes από την αρχή του partition και έχει μήκος 1024 bytes.

- Αν το μέγεθος του μπλοκ είναι 1KB (1024 bytes), το Superblock βρίσκεται στο Μπλοκ 1.
- Αν το μέγεθος του μπλοκ είναι μεγαλύτερο (π.χ. 2KB ή 4KB), το Superblock βρίσκεται μέσα στο Μπλοκ 0 (μετά τα πρώτα 1024 bytes που δεσμεύονται για το Boot Sector).

15) Η ύπαρξη εφεδρικών αντιγράφων είναι ζωτικής σημασίας για την ανάκτηση δεδομένων και την ανθεκτικότητα του συστήματος. Επειδή το Superblock είναι απαραίτητο για το mount του δίσκου, αν το πρωτεύον Superblock καταστραφεί (π.χ. λόγω κακού τομέα στον δίσκο ή σφάλματος εγγραφής), ολόκληρο το σύστημα αρχείων θα γινόταν μη αναγνώσιμο. Τα εφεδρικά αντίγραφα επιτρέπουν στο εργαλείο επισκευής fsck (File System Check) να ανακτήσει τη δομή του δίσκου χρησιμοποιώντας ένα από τα αντίγραφα.

16) Για να βρούμε σε ποια block βρίσκονται αποθηκευμένα τα εφεδρικά αντίγραφα, αρκεί να ψάξουμε την παράμετρο Backup superblock με την dumpe2fs.

```
root@utopia:/home/user/shared/ext2/ext2-vdisks# dumpe2fs /dev/vdb | grep "superblock"
dumpe2fs 1.47.0 (5-Feb-2023)
Primary superblock at 1, Group descriptors at 2-2
Backup superblock at 8193, Group descriptors at 8194-8194
Backup superblock at 16385, Group descriptors at 16386-16386
Backup superblock at 24577, Group descriptors at 24578-24578
Backup superblock at 32769, Group descriptors at 32770-32770
Backup superblock at 40961, Group descriptors at 40962-40962
Backup superblock at 49153, Group descriptors at 49154-49154
```

Για να βρούμε σε ποια block βρίσκονται τα εφεδρικά αντίγραφα με hexedit, αρκεί να βρούμε πόσα blocks per group υπάρχουν και να πάρουμε το πολλαπλάσιο του αριθμού αυτού ως το πρώτο μπλοκ κάθε ομάδας.

Το offset 0x420 (*s_blocks_per_group*) έχει την τιμή 00 20 00 00 (0x2000), δηλαδή 8192 blocks ανά ομάδα.

Επειδή το First Data Block είναι το 1 (στο offset 0x414 ως 01 00 00 00), οι υπολογισμοί είναι:

- Group 1: $1 + (1 * 8192) = 8.193$
- Group 2: $1 + (2 * 8192) = 16.385$
- Group 3: $1 + (3 * 8192) = 24.577$
- Group 4: $1 + (4 * 8192) = 32.769$
- Group 5: $1 + (5 * 8192) = 40.961$
- Group 6: $1 + (5 * 8192) = 49.153$

Το επόμενο θα ήταν στο Group 7 -> 57.345, αλλά ο δίσκος μας έχει μόνο 51.200 μπλοκ, οπότε σταματάμε εδώ.

17) Το Block Group (Ομάδα Μπλοκ) είναι μια μέθοδος οργάνωσης που χρησιμοποιεί το σύστημα αρχείων ext2 για να χωρίσει τον φυσικό δίσκο σε μικρότερες, λογικές ενότητες (partitions within a partition). Σκοπός του είναι να μειώσει τον κατακερματισμό (fragmentation) και να αυξήσει την ταχύτητα ανάγνωσης.

Κάθε Block Group λειτουργεί σαν ένα μικρό αυτόνομο σύστημα αρχείων που περιέχει τα δικά του αντίγραφα του Superblock, τους πίνακες περιγραφής (Group Descriptors), τα Bitmaps (για inodes και blocks), τον πίνακα Inodes και φυσικά τα δεδομένα. Το σύστημα προσπαθεί να αποθηκεύει τα δεδομένα ενός αρχείου στο ίδιο block group με το inode του, ώστε η κεφαλή του δίσκου να μην χρειάζεται να μετακινείται μακριά (seek time).

18) Ο αριθμός των block groups σε ένα σύστημα αρχείων ext2 **δεν είναι σταθερός**, αλλά καθορίζεται δυναμικά κατά τη δημιουργία του συστήματος αρχείων. Εξαρτάται από δύο βασικούς παράγοντες:

1. Το συνολικό μέγεθος του δίσκου (ή του partition).
2. Το ορισμένο μέγεθος του μπλοκ (block size).

Ο αριθμός τους υπολογίζεται διαιρώντας το **Σύνολο των Μπλοκ** με τον αριθμό των **Μπλοκ ανά Group** (Total Blocks / Blocks per Group).

Όσον αφορά την κατανομή τους, τα block groups τοποθετούνται **διαδοχικά** (sequentially) στον φυσικό χώρο του δίσκου (π.χ. Group 0, Group 1, Group 2 κ.λπ.), ξεκινώντας από την αρχή και καλύπτοντας όλο τον διαθέσιμο χώρο αποθήκευσης.

19) Βρίσκουμε τα συνολικά Block groups, κάνοντας pipe όλες τις γραμμές που επιστρέφει η duple2fs για την παράμετρο Group στην wc -l, που επιστρέφει τον συνολικό αριθμό γραμμών.

```
root@utopia:/home/user/shared/ext2/ext2-vdisks# duple2fs /dev/vdb | grep "^Group" | wc -l
duple2fs 1.47.0 (5-Feb-2023)
7
```

Ο αριθμός των block groups υπολογίζεται ως: *Total blocks / Blocks per group*.

Total Blocks (*s_blocks_count*): Στο offset 0x404 βρήκαμε την τιμή 51.200 (Ερ.2)

Blocks per Group (*s_blocks_per_group*): Στο offset 0x420 βρήκαμε την τιμή 8.192 (Ερ.16)

Υπολογίζουμε $51.200 / 8.192 = 6,25$. Επειδή το αποτέλεσμα δεν είναι ακέραιος, σημαίνει ότι έχουμε 6 πλήρεις ομάδες και χρειαζόμαστε άλλη μία για τα υπόλοιπα δεδομένα. Επομένως, έχουμε 7 Block Groups (αριθμημένα από Group 0 έως Group 6)

20) Ο **Block Group Descriptor** είναι μια δομή δεδομένων που περιγράφει τα χαρακτηριστικά και τη θέση των μεταδεδομένων για κάθε Block Group. Στην ουσία, λειτουργεί ως "ευρετήριο" για την ομάδα. Κάθε descriptor περιέχει δείκτες (pointers) που λένε στο σύστημα πού ακριβώς ξεκινούν μέσα στο συγκεκριμένο Group:

- Ο πίνακας Block Bitmap (χάρτης ελεύθερων μπλοκ).
- Ο πίνακας Inode Bitmap (χάρτης ελεύθερων inodes).
- Ο πίνακας Inode Table (πίνακας των inodes). Επιπλέον, καταγράφει στατιστικά στοιχεία, όπως τον αριθμό των ελεύθερων μπλοκ και inodes στη συγκεκριμένη ομάδα. Όλοι οι descriptors μαζεύονται σε έναν ενιαίο πίνακα που ονομάζεται Group Descriptor Table (GDT).

21) Τα εφεδρικά αντίγραφα είναι κρίσιμα για την ακεραιότητα και την ανάκτηση του συστήματος αρχείων. Επειδή ο πίνακας περιγραφών (GDT) είναι αυτός που "δείχνει" πού βρίσκονται τα inodes και τα bitmaps, αν καταστραφεί το πρωτεύον αντίγραφο, το λειτουργικό σύστημα χάνει την πρόσβαση στη δομή των αρχείων. Τα αντίγραφα επιτρέπουν στο εργαλείο fsck να αποκαταστήσει τον πίνακα και να σώσει τα δεδομένα σε περίπτωση φθοράς του δίσκου.

22) Στο σύστημα αρχείων ext2, τα αντίγραφα των Group Descriptors βρίσκονται ακριβώς στο επόμενο μπλοκ μετά από κάθε Backup Superblock.

```
root@utopia:/home/user/shared/ext2/ext2-vdisks# dumpe2fs /dev/vdb | grep "Group descriptors"
dumpe2fs 1.47.0 (5-Feb-2023)
Primary superblock at 1, Group descriptors at 2-2
Backup superblock at 8193, Group descriptors at 8194-8194
Backup superblock at 16385, Group descriptors at 16386-16386
Backup superblock at 24577, Group descriptors at 24578-24578
Backup superblock at 32769, Group descriptors at 32770-32770
Backup superblock at 40961, Group descriptors at 40962-40962
Backup superblock at 49153, Group descriptors at 49154-49154
```

Στην Ερώτηση 16 με hexedit βρήκαμε ότι τα αντίγραφα του Superblock βρίσκονται στα μπλοκ 8.193, 16.385, 24.577, 32.769, 40.961 και 49.153. Επομένως, οι πίνακες περιγραφών (Group Descriptor Tables) ακολουθούν ακριβώς μετά:

- Group 1: $8.193 + 1 = 8.194$
- Group 2: $16.385 + 1 = 16.386$
- Group 3: $24.577 + 1 = 24.578$
- Group 4: $32.769 + 1 = 32.770$

- Group 5: $40.961 + 1 = 40.962$
- Group 6: $49.153 + 1 = 49.154$

23)

- **Block Bitmap (Χάρτης ψηφίων μπλοκ):** Είναι μια δομή δεδομένων που λειτουργεί ως ένας "πίνακας ελέγχου" για τα μπλοκ δεδομένων. Κάθε bit σε αυτόν τον χάρτη αντιστοιχεί σε ένα συγκεκριμένο μπλοκ στον δίσκο.
 - Αν το bit είναι 0, το μπλοκ είναι ελεύθερο.
 - Αν το bit είναι 1, το μπλοκ είναι δεσμευμένο (χρησιμοποιείται). Έτσι, το σύστημα βρίσκει γρήγορα πού υπάρχει κενός χώρος για να γράψει δεδομένα.
- **Inode Bitmap (Χάρτης ψηφίων inode):** Λειτουργεί με την ίδια ακριβώς λογική, αλλά για τα inodes. Κάθε bit αντιστοιχεί σε ένα inode στον πίνακα inodes (Inode Table). Το σύστημα το συμβουλεύεται για να βρει ένα ελεύθερο inode όταν δημιουργείται ένα νέο αρχείο ή φάκελος.

Αυτές οι δομές δεν βρίσκονται όλες μαζεμένες σε ένα σημείο, αλλά είναι κατανεμημένες. Κάθε Block Group περιέχει το δικό του Block Bitmap και Inode Bitmap, τα οποία διαχειρίζονται τα μπλοκ και τα inodes της συγκεκριμένης ομάδας.

Συγκεκριμένα:

```
root@utopia:/home/user/shared/ext2/ext2-vdisks# dumpe2fs /dev/vdb | grep -E "Block bitmap|Inode bitmap"
dumpe2fs 1.47.0 (5-Feb-2023)
Block bitmap at 3 (+2)
Inode bitmap at 4 (+3)
Block bitmap at 8195 (+2)
Inode bitmap at 8196 (+3)
Block bitmap at 16387 (+2)
Inode bitmap at 16388 (+3)
Block bitmap at 24579 (+2)
Inode bitmap at 24580 (+3)
Block bitmap at 32771 (+2)
Inode bitmap at 32772 (+3)
Block bitmap at 40963 (+2)
Inode bitmap at 40964 (+3)
Block bitmap at 49155 (+2)
Inode bitmap at 49156 (+3)
```

Για να βρούμε πού βρίσκονται μέσα στον δίσκο το block bitmap και το inode bitmap, θα βρούμε το Group Descriptor Table (GDT). Το GDT ξεκινάει στο Block 2 (αμέσως μετά το κύριο Superblock του Block 1). Εφόσον το Block Size είναι 1024 bytes (Ερ. 9), το Block 2 ξεκινάει στο offset: $2 * 1024 = 2048$ (0x800).

Block Bitmap (0x800): Βλέπουμε 03 00 00 00. Αρα βρίσκεται στο Block 3 του κάθε block group.

Inode Bitmap (0x804): Βλέπουμε 04 00 00 00. Αρα βρίσκεται στο Block 4 του κάθε block group.

24) Ο **Πίνακας Inode (Inode Table)** είναι η δομή δεδομένων που περιέχει τις πραγματικές εγγραφές των inodes. Είναι ένα array όπου κάθε γραμμή περιέχει όλες τις πληροφορίες (metadata) για ένα συγκεκριμένο αρχείο ή φάκελο. Λόγου χάριν, όταν το σύστημα αναφέρεται στο "inode 12", ουσιαστικά πηγαίνει στη 12η θέση αυτού του πίνακα για να διαβάσει τα στοιχεία του αρχείου.

Όπως και τα bitmaps, ο πίνακας Inode Table δεν βρίσκεται σε ένα μόνο σημείο, αλλά είναι κατανεμημένος (distributed) σε όλα τα Block Groups του δίσκου. Κάθε Block Group έχει το δικό του τμήμα του Inode Table, το οποίο περιέχει τα inodes που ανήκουν στη συγκεκριμένη ομάδα.

Μέσα σε ένα Block Group, ο πίνακας Inode Table βρίσκεται αμέσως μετά το Inode Bitmap και καταλαμβάνει έναν σημαντικό αριθμό συνεχόμενων μπλοκ (ανάλογα με το πόσα inodes έχουν οριστεί ανά group).

```
root@utopia:/home/user/shared/ext2/ext2-vdisks# dumpe2fs /dev/vdb | grep "Inode table"
dumpe2fs 1.47.0 (5-Feb-2023)
Inode table at 5-233 (+4)
Inode table at 8197-8425 (+4)
Inode table at 16389-16617 (+4)
Inode table at 24581-24809 (+4)
Inode table at 32773-33001 (+4)
Inode table at 40965-41193 (+4)
Inode table at 49157-49385 (+4)
```

Ο πίνακας των inodes βρίσκεται αμέσως μετά το inode bitmap, δηλαδή στο 5ο Block του κάθε Block Group.

25) Το inode (index node) είναι η δομή που περιέχει όλα τα μεταδεδομένα ενός αρχείου, εκτός από το όνομά του. Τα βασικά πεδία που περιλαμβάνει είναι:

- **Mode (Τύπος & Δικαιώματα):** Καθορίζει αν είναι αρχείο, κατάλογος ή link, και ποια δικαιώματα (r/w/x) έχουν οι χρήστες.
- **Owner Info (Ιδιοκτησία):** Το User ID (UID) και Group ID (GID) του ιδιοκτήτη.
- **Size (Μέγεθος):** Το μέγεθος του αρχείου σε bytes.
- **Timestamps (Χρονοσφραγίδες):** Πότε δημιουργήθηκε, πότε προσπελάστηκε (atime), πότε τροποποιήθηκε (mtime) και πότε διαγράφηκε (dtime).
- **Link Count:** Πόσα hard links δείχνουν σε αυτό το αρχείο.
- **Data Block Pointers (Δείκτες Δεδομένων):** Περιέχει 15 δείκτες που δείχνουν σε ποια μπλοκ του δίσκου είναι αποθηκευμένο το περιεχόμενο του αρχείου (12 άμεσοι, 1 έμμεσος, 1 διπλά έμμεσος, 1 τριπλά έμμεσος).

Το κάθε inode αποθηκεύεται ως μια εγγραφή μέσα στον Πίνακα Inode (Inode Table) του Block Group στο οποίο ανήκει. Κάθε inode έχει ένα μοναδικό αριθμό (Inode Number). Το σύστημα χρησιμοποιεί αυτόν τον αριθμό ως δείκτη (index) για να εντοπίσει τη συγκεκριμένη θέση του inode μέσα στον πίνακα Inode Table.

26) Με τις παραμέτρους Blocks per group και Inodes per group με τη χρήση της dumpe2fs βρήκαμε πόσα μπλοκ και πόσα inodes περιέχει το κάθε block group σε αυτό το σύστημα αρχείων.

```
root@utopia:/home/user/shared/ext2/ext2-vdisks# dumpe2fs /dev/vdb | grep -E "Blocks per group|Inodes per group"
dumpe2fs 1.47.0 (5-Feb-2023)
Blocks per group:      8192
Inodes per group:      1832
```

Με τη χρήση hexedit:

```
00000420  00 20 00 00 00 20 00 00 28 07 00 00
```

Μπλοκ ανά ομάδα (*s_blocks_per_group*): Βρίσκεται στο offset 0x420, όπου βλέπουμε την τιμή 00 20 00 00. $0x2000 = 2 * 16^3 = 8.192$ blocks.

Inodes ανά ομάδα (*s_inodes_per_group*): Βρίσκεται στο offset 0x428, όπου βλέπουμε την τιμή 28 07 00 00. Είναι $0x0728 = 7 * 256 + 2 * 16 + 8 = 1792 + 32 + 8 = 1.832$ inodes.

27) Με την `ls -li` βρήκαμε το inode του αρχείου στο `/mnt/utopia/fs1/dir2/helloworld`.

```
root@utopia:/home/user/shared/ext2/ext2-vdisks# ls -li /mnt/fs1/dir2/helloworld
9162 /mnt/fs1/dir2/helloworld
```

Η ρίζα του συστήματος αρχείων είναι πάντα το Inode 2. Πρέπει να δούμε σε ποιο μπλοκ αποθηκεύει τα δεδομένα του (τα ονόματα των αρχείων/φακέλων που περιέχει). Ο Πίνακας Inodes ξεκινάει στο Block 5, δηλαδή στο offset 0x1400 (όπως βρήκαμε πριν). Αφού το μέγεθος είναι 128 bytes έχουμε Inode 1: 0x1400 Inode 2: 0x1480 κλπ.

Τον δείκτη για το πρώτο Data Block (*i_block[0]*) τον βρίσκουμε στο offset 0x28 (40 bytes) μέσα στο inode. Άρα ψάχνουμε στη διεύθυνση: $0x1480 + 0x28 = 0x14A8$. Το μπλοκ δεδομένων είναι το 0xEA = 234, που είναι δείκτης στον dir2.

Για να δούμε τι περιέχει, πρέπει να πάμε στη διεύθυνση: $234 * 1024 = 239616$ (0x3A800), όπου βρίσκουμε το inode του dir2/helloworld: C9 23 00 00 (0x23C9 ή 9161).

28) Ο αριθμός του Block Group προκύπτει αν διαιρέσουμε τον αριθμό του inode (μειωμένο κατά 1, καθώς η αρίθμηση των groups ξεκινά από το 0) με το πλήθος των inodes που περιέχει κάθε group.

Συγκεκριμένα: $(9162 - 1) / 1832 = 9161 / 1832 = 5,0005...$

Κρατάμε το ακέραιο μέρος του αποτελέσματος, άρα το αρχείο βρίσκεται στο Group 5.

29) Χρησιμοποιούμε την dumpe2fs για να δούμε την δομή του Block Group 5:

```
root@utopia:/home/user/shared/ext2/ext2-vdisks# dumpe2fs /dev/vdb | grep -A 7 "Group 5:"
dumpe2fs 1.47.0 (5-Feb-2023)
Group 5: (Blocks 40961-49152)
  Backup superblock at 40961, Group descriptors at 40962-40962
  Block bitmap at 40963 (+2)
  Inode bitmap at 40964 (+3)
  Inode table at 40965-41193 (+4)
  7959 free blocks, 1830 free inodes, 1 directories
  Free blocks: 41194-49152
  Free inodes: 9163-10992
```

Το inode table που περιέχει το συγκεκριμένο inode ξεκινάει στο μπλοκ 40965 και εκτείνεται μέχρι το μπλοκ 41193.

Με hexedit, αφού βρήκαμε πως το inode βρίσκεται στο Group 5, αρκεί να βρούμε το offset του Inode table του group 5. Στο ερ. 24 βρήκαμε 40.965 - 41.193.

30) Κάνοντας debugfs -R "stat <9162>" fsdisk1-7c2b425a6.img βλέπουμε όλα τα πεδία του inode 9162:

```
Inode: 9162    Type: regular    Mode: 0644    Flags: 0x0
Generation: 2739270588    Version: 0x00000001
User:      0    Group:      0    Size: 42
File ACL: 0
Links: 1    Blockcount: 2
Fragment: Address: 0    Number: 0    Size: 0
ctime: 0x65787ae4 -- Tue Dec 12 15:23:16 2023
atime: 0x65787ae4 -- Tue Dec 12 15:23:16 2023
mtime: 0x65787ae4 -- Tue Dec 12 15:23:16 2023
BLOCKS:
(0):1025
TOTAL: 1
```

Για να δούμε τα πεδία του inode με hexedit, αρκεί να μεταβούμε στο offset:

$inode\ offset = inode\ table\ start + (inode\ number - 1) * inode\ size$

Είναι: $inode\ offset = 40.965 * 1024 + (9162 - 1) * 128 = 43.107.328 = 0x294C800$.

31) Στην έξοδο της εντολής debugfs, το πεδίο BLOCKS αναφέρει: (0):1025. Αυτό σημαίνει ότι το 1ο μπλοκ δεδομένων του αρχείου (λογική θέση 0) αντιστοιχεί στο φυσικό μπλοκ 1025 του δίσκου.

32) Το μέγεθος του αρχείου είναι 42 bytes.

fdisk2.img

- 1) Συνδέσαμε την εικόνα fdisk2.img στην εικονική μας μηχανή και την προσαρτήσαμε στον κατάλογο /mnt όπως παρακάτω:

```
root@utopia:/home/user/shared/ext2/ext2-vdisks# mkdir -p /mnt/fs2
root@utopia:/home/user/shared/ext2/ext2-vdisks# mount /dev/vdc /mnt/fs2
root@utopia:/home/user/shared/ext2/ext2-vdisks# ls -l /mnt/fs2
total 25
drwxr-xr-x 22 root root 1024 Dec 12 2023 dir-0
drwxr-xr-x 22 root root 1024 Dec 12 2023 dir-1
drwxr-xr-x 22 root root 1024 Dec 12 2023 dir-10
drwxr-xr-x 22 root root 1024 Dec 12 2023 dir-11
drwxr-xr-x 6 root root 1024 Dec 12 2023 dir-12
drwxr-xr-x 22 root root 1024 Dec 12 2023 dir-2
drwxr-xr-x 22 root root 1024 Dec 12 2023 dir-3
drwxr-xr-x 22 root root 1024 Dec 12 2023 dir-4
drwxr-xr-x 22 root root 1024 Dec 12 2023 dir-5
drwxr-xr-x 22 root root 1024 Dec 12 2023 dir-6
drwxr-xr-x 22 root root 1024 Dec 12 2023 dir-7
drwxr-xr-x 22 root root 1024 Dec 12 2023 dir-8
drwxr-xr-x 22 root root 1024 Dec 12 2023 dir-9
drwx----- 2 root root 12288 Dec 12 2023 lost+found
root@utopia:/home/user/shared/ext2/ext2-vdisks#
```

- 2) Έγινε προσπάθεια δημιουργίας νέου αρχείου file1 στον κατάλογο /mnt/utopia/fs2 με την touch.

```
root@utopia:/home/user/shared/ext2/ext2-vdisks# touch /mnt/utopia/fs2/file1
touch: cannot touch '/mnt/utopia/fs2/file1': No space left on device
```

- 3) Η εντολή απέτυχε. Το μήνυμα λάθους που επέστρεψε το shell είναι:

touch: cannot touch '/mnt/utopia/fs2/file1': No space left on device

Αυτό σημαίνει ότι το σύστημα αρχείων δεν έχει διαθέσιμο χώρο για τη δημιουργία νέου αρχείου. Επειδή η εντολή touch δημιουργεί ένα αρχείο 0 bytes (κενό), αυτό συνήθως υποδηλώνει ότι έχουν εξαντληθεί τα Inodes (οι θέσεις για τα αρχεία) και όχι απαραίτητα ο χώρος δεδομένων (Data Blocks).

4) Με την strace για την touch, έχουμε:

```
openat(AT_FDCWD, "/mnt/utopia/fs2/file1", O_WRONLY|O_CREAT|O_NOCTTY|O_NONBLOCK, 0666) = -1 ENOSPC (No space left on device)
```

Η εντολή touch προσπάθησε να εκτελέσει την κλήση συστήματος openat() για να δημιουργήσει το αρχείο. Η κλήση απέτυχε επιστρέφοντας την τιμή -1 και τον κωδικό σφάλματος ENOSPC (που αντιστοιχεί στο μήνυμα "No space left on device").

5) Με τη χρήση της find και κάνοντας grep στην wc -l όπως πριν, λαμβάνουμε τον συνολικό αριθμό αρχείων και τον συνολικό αριθμό καταλόγων του συστήματος αρχείων.

```
root@utopia:/home/user/shared/ext2/ext2-vdisks# find /mnt/utopia/fs2 -type f | wc -l
4868
root@utopia:/home/user/shared/ext2/ext2-vdisks# find /mnt/utopia/fs2 -type d | wc -l
259
```

Στο primary superblock:

- s_log_block_size (offset 0x18): 00000414 01 00 00 00 00 00 00 00, άρα ένα block είναι 1024 bytes.
- s_inode_size (offset 0x58): 00000454 0B 00 00 00 80 00 00 00, άρα ένα inode έχει 128 bytes.
- s_blocks_count (offset 0x04): 00000400 10 14 00 00 00 50 00 00, άρα έχει συνολικά 20.480 blocks.
- s_inodes_count (offset 0x00): έχει 5136 inodes.
- s_blocks_per_group (offset 0x20): 0x2000 (8192 blocks per group)
- s_inodes_per_group (offset 0x28): 0x06B0 (1712 inodes per group)

```
00000420 00 20 00 00 00 20 00 00 B0 06 00 00
```

Αφού έχουμε 8192 μπλοκ ανά ομάδα, και αφού το σύστημα αρχείων έχει μέγεθος 0x1400000 = 20971520 bytes (20MB), δηλαδή περιέχει το πολύ 20.480 blocks, έχει τα εξής block groups:

- Group 1: 0 έως 8191
- Group 2: 8192 + έως $(2 * 8192 - 1) = 16.385$
- Group 3: 16.386 έως 20.480 (όριο)

Ο πίνακας των inodes βρίσκεται στο 5ο Block του κάθε Block Group. Για να γίνει μέσω hexedit ο υπολογισμός των inode files και των inode directories, θα έπρεπε να ελέγχουμε το πεδίο i_mode (τα πρώτα 2 bytes του inode) για να δούμε εάν πρόκειται για file (0x8000) ή directory (0x4000), για καθένα από τα 5136 inodes.

6) Για να βρούμε τον χώρο που καταλαμβάνουν τα δεδομένα και τα μεταδεδομένα, αρκεί να προσδιορίσουμε τον αριθμό των blocks που καταλαμβάνουν τα μεταδεδομένα και να τον αφαιρέσουμε από τον συνολικό αριθμό των blocks για να λάβουμε τον αριθμό των data blocks.

Η εικόνα περιέχει συνολικά 20.480 blocks. Έχει 3 block groups.

Κάθε block group θα περιέχει τα εξής blocks για metadata:

- 1 block για το superblock
- 1 block για το block group descriptor table
- 1 block για το block bitmap
- 1 block για το inode bitmap
- $s_inodes_per_group * s_inode_size / block_size = 214$ blocks για το inode table

Δηλαδή κάθε block group περιέχει 218 blocks για metadata, πέραν του boot block στην αρχή του συστήματος αρχείων. Αφού κάθε block group περιέχει 8192 blocks, έχει 7974 data blocks. Αφού το σύστημα αρχείων απαρτίζεται από 3 block groups, έχουμε συνολικά $1 + 3 * 214 = 642$ metadata blocks και $20.480 - 642 = 19.837$ data blocks.

7) Στο superblock έχουμε:

- `s_log_block_size`: 1024 bytes
- `s_blocks_count` (offset 0x04): 0x5000 = 20480 blocks

Άρα το συνολικό μέγεθος του συστήματος αρχείων είναι: $1024 * 20.480 = 20.971.520$ bytes = 20 MB.

```
root@utopia:/home/user/shared/ext2/ext2-vdisks# df -h /mnt/utopia/fs2
Filesystem      Size  Used Avail Use% Mounted on
/dev/loop1      20M   270K   20M   2% /mnt/utopia/fs2
root@utopia:/home/user/shared/ext2/ext2-vdisks# df -i /mnt/utopia/fs2
Filesystem      Inodes IUsed IFree IUse% Mounted on
/dev/loop1      5136  5136    0 100% /mnt/utopia/fs2
```

8) Το συνολικό μέγεθος του συστήματος αρχείων είναι 20 MB. Το σύστημα αρχείων έχει άφθονο ελεύθερο χώρο αποθήκευσης δεδομένων. Συγκεκριμένα, υπάρχουν διαθέσιμα 20 Megabytes (στήλη Avail), ενώ έχει χρησιμοποιηθεί μόλις το 2% του συνολικού χώρου (270K).

Με hexedit, κοιτάμε το πεδίο `s_free_blocks_count` (με offset 0x0C από την αρχή του superblock): **0000040C 63 4C 00 00**, που σε little endian είναι 0x4C63. Σε δεκαδικό είναι 19.555, δηλαδή 19.555 blocks είναι διαθέσιμα.

9) Στα συστήματα αρχείων τύπου UNIX (όπως το ext2), για να δημιουργηθεί ένα αρχείο απαιτούνται δύο πόροι:

1. Data Blocks: Για να αποθηκευτούν τα δεδομένα (περιεχόμενο).
2. Ένα Inode: Για να αποθηκευτούν τα μεταδεδομένα (ταυτότητα αρχείου, δικαιώματα, δείκτες).

Στην προκειμένη περίπτωση αυτό που αποτρέπει τη δημιουργία νέου αρχείου είναι η εξάντληση των Inodes (**Inode Exhaustion**).

Στο superblock βρίσκουμε τα πεδία:

000000400	10	14	00	00	00	50	00	00
00000040C	63	4C	00	00	00	00	00	00

s_inodes_count (offset 0x00): 0x1410, δηλαδή υπάρχουν 5.136 inodes.

s_free_inodes_count (0x10): 0x0000, δηλαδή κανένα από αυτά δεν είναι διαθέσιμο.

Συνεπώς, το σύστημα περιέχει 5.136 αρχεία και καταλόγους, παρότι περιέχει ελεύθερο χώρο σε MB.

fsdisk3.img

- 1) Το εργαλείο στο Linux που αναλαμβάνει τον έλεγχο ενός συστήματος αρχείων ext2 για αλλοιώσεις είναι το fsck.ext2. Το fsck διαβάζει απευθείας τις δομές metadata του filesystem και ελέγχει τη συνέπεια μεταξύ τους.

Το superblock του συστήματος αρχείων περιέχει πεδία που υποδεικνύουν εάν πρέπει να τρέξει η fsck (consistency check) όταν γίνεται boot. Η fsck θα τρέξει εάν το σύστημα δεν έγινε σωστό mount του συστήματος, εάν έχει ξεπεράσει το μέγιστο πλήθος mount, ή εάν ο μέγιστος χρόνος μεταξύ ελέγχων έχει υπερβεί.

- 2) Αλλοιώσεις στο σύστημα αρχείων μπορεί να προκληθούν από hardware και software failures.

Ενδεικτικά:

- Δυσλειτουργία τομέων, οπότε υπάρχουν data sections που δεν μπορούν αξιόπιστα να αποθηκεύσουν δεδομένα.
- Read/write heads που δεν μπορούν αξιόπιστα να προσπελάσουν τους τομείς.
- Φθορά της συσκευής driver λόγω παρατεταμένης χρήσης.
- Transmission errors λόγω ακατάλληλου disk interface (SATA/IDE controllers, USB connections, PCIe slots και καλώδια) μεταξύ host και συσκευής αποθήκευσης δεδομένων.
- Ζημιά στο μέσο αποθήκευσης δεδομένων (π.χ. γρατζουνιά σε CD/DVD).
- Αποσύνδεση του μέσου αποθήκευσης δεδομένων ενώ γίνεται εγγραφή δεδομένων (π.χ. αφαίρεση USB ενώ γίνεται εγγραφή δεδομένων).
- Shutdown πριν γίνει εγγραφή των δεδομένων που μεταβλήθηκαν στο σύστημα αρχείων, με το journaling απενεργοποιημένο. (Το journal είναι ένα αρχείο στο οποίο καταγράφονται τα blocks που έχουν αλλοιωθεί πριν εγγραφούν οι αλλαγές στο σύστημα αρχείων.)
- Firmware bugs στις I/O ρουτίνες του λειτουργικού συστήματος.
- Malware που επιτρέπει την κακόβουλη πρόσβαση στη μνήμη και άρα την κωδικοποίηση ή διαγραφή δεδομένων.
- Ακατάλληλη υλοποίηση του συστήματος αρχείων όπως κακό error handling ή edge cases που οδηγούν σε bugs. Παράδειγμα το EXT4 fsync bug (μεγάλο interval μεταξύ saves).

3) Τρέξαμε την fsck.ext2 στο fsdisk3.img, όπως φαίνεται παρακάτω:

```
root@utopia:/home/user/shared/ext2/ext2-vdisks# fsck.ext2 -f -v /dev/vdd
e2fsck 1.47.0 (5-Feb-2023)
Pass 1: Checking inodes, blocks, and sizes
Pass 2: Checking directory structure
First entry 'BOO' (inode=1717) in directory inode 1717 (/dir-2) should be '.'
Fix<y>? yes
Pass 3: Checking directory connectivity
Pass 4: Checking reference counts
Inode 3425 ref count is 1, should be 2. Fix<y>? yes
Pass 5: Checking group summary information
Block bitmap differences: +34
Fix<y>? yes
Free blocks count wrong (926431538, counted=19800).
Fix<y>? yes

fsdisk3.img: ***** FILE SYSTEM WAS MODIFIED *****

    23 inodes used (0.45%, out of 5136)
    0 non-contiguous files (0.0%)
    0 non-contiguous directories (0.0%)
    # of inodes with ind/dind/tind blocks: 0/0/0
  680 blocks used (3.32%, out of 20480)
    0 bad blocks
    0 large files

    9 regular files
    5 directories
    0 character device files
    0 block device files
    0 fifos
    0 links
    0 symbolic links (0 fast symbolic links)
    0 sockets
-----
   14 files
```

Έγιναν οι παρακάτω αλλαγές:

- Στο directory inode 1717 το πρώτο entry 'BOO' μεταβλήθηκε σε '.' Αυτό έγινε αφού το πρώτο entry κάθε directory πρέπει να είναι ο αυτός του.
- Το reference count του inode 3425 ήταν 1 ενώ ανιχνεύθηκαν 2 directory entries σε αυτό. Επομένως, μεταβλήθηκε σε 2.
- Βρέθηκαν 34 block bitmap λάθη, δηλαδή 34 blocks χρησιμοποιούνταν ενώ ήταν marked ως free. Η fsck το διόρθωσε.
- Το free blocks count ήταν 926.431.538 ενώ μετρήθηκαν 19.800 διαθέσιμα μπλοκ.

4) Με τη χρήση της hexedit, εντοπίσαμε τις παραπάνω αλλοιώσεις στην αρχική εικόνα.

Αρχικά ελέγχουμε το primary superblock:

- s_inodes_count (offset 0x00): **000003FC 00 00 00 00 10 14 00 00**
άρα έχει συνολικά 5.136 inodes.
- s_blocks_count (offset 0x04): **00000404 00 50 00 00** άρα έχει συνολικά 20.480 blocks.
- s_log_block_size(offset 0x18): **00000414 01 00 00 00 00 00 00 00**
άρα ένα block είναι 1024 bytes.
- s_blocks_per_group (offset 0x20): (0x2000) 8.192 blocks per group.
00000420 00 20 00 00 00 20 00 00 B0 06 00 00
- s_inodes_per_group (offset 0x28): (0x06B0) 1712 inodes per group
- s_inode_size (offset 0x58): **00000450 00 00 00 00 0B 00 00 00 80 00 00 00**,
άρα ένα inode έχει 128 bytes.

Εύρεση πρώτου entry του directory node 1717

Αρκεί να βρούμε το πρώτο direct pointer από τα μεταδεδομένα του inode.

Το block group του inode 1717 είναι το:

$block\ group = (inode\ number - 1) / inodes\ per\ group = 1716 / 1712 = 1.002...$
Ανήκει στο block group 1, άρα θα προσθέσουμε offset το s_blocks_per_group *
block_size = 8192 * 1024 = 0x800000.

Από το block group descriptor θα λάβουμε το offset του inode table (bg_inode_table), που βρίσκεται στο offset 0x08 από την αρχή του block group descriptor. Βρίσκεται αμέσως μετά το superblock, το οποίο έχει μέγεθος 1024 bytes, οπότε μας ενδιαφέρει το offset από την αρχή της εικόνας 0x800808.

00800800 03 00 00 00 04 00 00 00 05 00 00 00

Είναι bg_inode_table = 0x05, δηλαδή βρίσκεται στο block 5. Άρα από την αρχή του block group έχει offset 5 * block_size = 5 * 1024 = 5.120 bytes = 0x1400.

Το offset του inode εντός του inode block είναι:

$index\ in\ group * inode\ size = 4 * 128 = 512\ bytes = 0x200$, όπου

$index\ in\ group = inode\ number - (group\ number * inodes\ per\ group) - 1 = 4$

Άρα η θέση του inode 1717 είναι $0x1400 + 0x200 + 0x800000 = 0x801600$. Αφού, επιπλέον ο pointer στο πρώτο directory entry του βρίσκεται 40 bytes μετά την αρχή του (0x28), θα δούμε τη διεύθυνση στη θέση 0x801628.

```
00801628  DC 20 00 00
```

Άρα το block number όπου βρίσκεται το πρώτο entry του inode 1717 είναι 0x20DC = block 8412. Είναι: $\text{byte offset} = \text{block number} * \text{block size} = 8412 * 1024 = 0x837000$.

```
00837000  B5 06 00 00  0C 00 03 00  42 4F 4F 00
```

Βρίσκουμε ως πρώτο entry τον ascii κώδικα 42 4F 4F, που είναι ισοδύναμος με 'BOO'.

Εύρεση του reference count του inode 3425

Αρκεί να βρούμε το inode 3425 και να διαβάσουμε το offset 0x1A (*i_links_count*). Το block group του inode 3425 είναι το:

$$\text{block group} = (\text{inode number} - 1) / \text{inodes per group} = 3425 / 1712 = 2,000...$$

Ανήκει στο block group 2, άρα θα προσθέσουμε offset το $2 * s_blocks_per_group * \text{block_size} = 2 * 8192 * 1024 = 0x1000000$.

```
01000808  05 00 00 00
```

Το inode table βρίσκεται πάλι στο block 5 όπως βλέπουμε παραπάνω, άρα έχει offset 0x1400 από την αρχή του block group.

Το offset του inode εντός του inode block είναι:

$$\text{index in group} * \text{inode size} = 0x00, \text{ όπου:}$$

$$\text{index in group} = \text{inode number} - (\text{group number} * \text{inodes per group}) - 1 = 0$$

Τελικά, η θέση του *i_links_count* του inode 3425 είναι απλά:

$$\text{block_group_offset} + \text{inode_table_offset} + \text{inode_offset} + \text{i_links_count_offset} =$$

$$0x1000000 + 0x1400 + 0x00 + 0x1A = 0x100141A$$

```
01001418  00 00 01 00
```

Βλέπουμε ότι έχει reference count 0x01.

Εύρεση διαφοράς block bitmaps στο block 34

To block 34 ανήκει στο block group 0. Το block bitmap βρίσκεται στο block 3 του group. Επομένως ελέγχουμε τη μνήμη στο offset 0xC00 έως την 0xFFFF. Στην αρχική εικόνα έχουμε:

[illegible]

Η fsck παράγαγε την ακόλουθη εικόνα, αφού έγινε έλεγχος όλων των 8192 blocks:

[illegible]

Βλέπουμε ότι όντως το bit 34 ήταν marked ως μη allocated, ενώ χρησιμοποιείται.

Εύρεση free blocks count

Το `s_free_blocks_count` έχει offset 12 bytes (0x0C) από την αρχή του superblock, οπότε ελέγχουμε το 0x40C.

```
00000400  10 14 00 00 00 50 00 00 00 04 00 00 32 39 38 37
00000410  F9 13 00 00 01 00 00 00 00 00 00 00 00 00 00 00
```

Αναγράφονται `0x37383932 = 926431538` free blocks, όπως επισήμανε η `fsck`.

5) Διορθώσαμε τις παραπάνω αλλοιώσεις με την `hexedit`:

- Διορθώσαμε το πρώτο entry του directory inode 1717 από 'BOO' σε '.', με ascii code 2E, και αλλάξαμε το μέγεθος σε 0x01, αντί του 0x03.

```
00837000  B5 06 00 00 0C 00 01 00 2E 00 00 00
```

- Διορθώσαμε το reference count του 3425 από 1 σε 2.

```
01001418  00 00 02 00
```

- Διορθώσαμε το block bitmap του block group 0, ώστε να είναι marked ως allocated το block 34.

```
00000C00  FF FF FF FF FF FF FF FF
```

- Διορθώσαμε το `s_free_blocks_count` από `0x37383932 = 926431538` σε 19800 = 4D58 = 58 4D 00 00 (little endian).

```
00000408  00 04 00 00 58 4D 00 00
```

Στο διορθωμένο αρχείο η `fsck` εκτελείται με `dry run`, δηλαδή δεν ανιχνεύει πλέον καμία ασυνέπεια.

```
root@utopia:/home/user/shared/ext2/ext2-vdisks# fsck.ext2 -v /dev/vdd
e2fsck 1.47.0 (5-Feb-2023)
fsdisk3.img: clean, 23/5136 files, 680/20480 blocks
```

2ο Μέρος: Το σύστημα αρχείων ext2-lite

Υλοποιήσαμε ένα απλουστευμένο ext2 σύστημα αρχείων, το ext2-lite, συμπληρώνοντας διάφορα κενά στον κώδικα που μας δίνεται έτσι ώστε το VFS να κάνει σωστά την αναγνώριση του. Ακολουθεί η επεξήγηση όλων των συμπληρωμένων συναρτήσεων.

init_ext2_fs / exit_ext2_fs

Με τις κλήσεις register_filesystem() και unregister_filesystem() δηλώνουμε στο VFS ότι υπάρχει filesystem με όνομα ext2-lite. Χωρίς αυτό, το insmod δεν μπορεί να βρει τον driver.

Στην συνάρτηση init_ext2_fs() που αρχικοποιεί το σύστημα αρχείων που προσαρτάται, καλούμε την register_filename με όρισμα τη δομή file_system_type *ext2_fs_type για να γίνει εισαγωγή του module στον πυρήνα. Η init καλείται όταν καλούμε την insmod.

Αντίστοιχα, όταν γίνεται αποπροσάρτηση του module αυτού με την rmmod, καλούμε την unregister_filename με την ίδια δομή ως παράμετρο.

```
static int __init init_ext2_fs(void)
{
    int err = init_inodecache();
    if (err)
        return err;

    /* Register ext2-lite filesystem in the kernel */
    err = register_filesystem(&ext2_fs_type);
    /* If an error occurs remember to call destroy_inodecache() */
    if (err < 0)
        destroy_inodecache();

    return err;
}
```

```
static void __exit exit_ext2_fs(void)
{
    /* Unregister ext2-lite filesystem from the kernel */
    unregister_filesystem(&ext2_fs_type);
    destroy_inodecache();
}
```

ext2_find_entry

Η συνάρτηση `ext2_find_entry()` σαρώνει τις directory entries ενός καταλόγου, σελίδα-σελίδα (folios) και επιστρέφει το inode entry που ταιριάζει με το ζητούμενο όνομα. Δέχεται ως παραμέτρους δείκτη στο directory inode, το child ως δομή `qstr` (qualified string) και δείκτη στον δείκτη foliop (page cache) ώστε να τον μεταβάλλει η συνάρτηση. Επιστρέφει pointer στο `ext2_dirent de` (directory entry).

- Με την `ext2_get_folio()` λαμβάνουμε καθεμία από τις σελίδες στην page cache του αντίστοιχου directory inode.
- Χρησιμοποιούμε την `ext2_last_byte()` για να ορίσουμε το εύρος των έγκυρων δεδομένων που ανήκουν στο directory. Επιστρέφει το τελευταίο χρησιμοποιούμενο byte της σελίδας `page_nr` του inode. Το όριο limit είναι ορισμένο έτσι ώστε να μην προσπελάσουμε πέραν του τέλους του directory.
- Στον βρόχο while συγκρίνουμε κάθε directory entry `de` με το ζητούμενο filename με την `ext2_match()` και σε περίπτωση που πετυχαίνει, το επιστρέφουμε, ενώ κάνουμε update το foliop με το νέο mapped folio, δηλαδή κάνουμε update την page cache.
- Εάν έχουμε φτάσει στο τέλος του directory, καλούμε την `folio_release_kmap()`, με την οποία μειώνεται το reference counter του folio, ώστε όταν δεν χρησιμοποιείται πια, να αποδεσμεύεται από την page cache.
- Τέλος, εάν η `ext2_find_entry` αποτύχει, καλούμε την `folio_release_kmap`, για να αποφύγουμε τα memory leaks.

```
/* Scan all the pages of the directory to find the requested name. */
for (i=0; i < npages; i++) {
    struct folio *folio;
    // returns address to the nth page of inode directory
    kaddr = ext2_get_folio(dir, i, 0, &folio);
    if (IS_ERR(kaddr))
        return ERR_CAST(kaddr);

    de = (ext2_dirent *)kaddr;
    char *limit = kaddr + ext2_last_byte(dir, i) - reclen;

    for (; (char *)de <= limit; de = ext2_next_entry(de)) {
        if (de->rec_len == 0) {
            folio_release_kmap(folio, kaddr);
            return ERR_PTR(-EIO);
        }

        if (ext2_match(namelen, name, de)) {
            *foliop = folio; // folio returned mapped
            return de;
        }
    }

    // call folio_release_kmap if ext2_find_entry fails
    folio_release_kmap(*foliop, kaddr);
}
```

ext2_get_inode

Η συνάρτηση `ext2_get_inode()` εντοπίζει ένα ext2 inode στο disk με βάση τον αριθμό του. Δέχεται ως παραμέτρους το superblock, από το οποίο λαμβάνει τις παραμέτρους που σχετίζονται με το σύστημα αρχείων, το inode number που παρέχει ο πυρήνας και δείκτη στον δείκτη buffer head, ο οποίος θα οριστεί στη συνάρτηση ως ο δείκτης στο block όπου βρίσκεται το inode, όπως υπολογίστηκε. Η συνάρτηση επιστρέφει τη θέση του δοσμένου inode στη φυσική μνήμη, ως το offset του inode number στο block που δείχνει ο buffer head.

- Υπολογίζουμε το block group στο οποίο ανήκει το inode ως $(\text{inode_number} - 1) / \text{inodes_per_group}$, και το χρησιμοποιήσαμε ως παράμετρο στην `ext2_get_group_descriptor` για να λάβουμε το group descriptor και άρα το inode table που του αντιστοιχεί.
- Αρχικοποιούμε το block ως το πρώτο block του inode table με την `le32_to_cpu(gdp->bg_inode_table)`.
- Το inode index είναι το $(\text{inode_number} - 1) \bmod \text{inodes_per_group}$. Λαμβάνουμε κατευθείαν το inode offset σε bytes ως $\text{inode index} * \text{inode size}$.
- Το ζητούμενο block από την αρχή του inode table είναι $\text{offset} / \text{blocksize}$. Τότε το offset σε bytes είναι $(\text{offset} \bmod \text{blocksize})$.
- Μετακινούμε το buffer head σε αυτό το block στην page cache με την `sb_bread()`. Εάν δεν βρίσκεται ήδη στην cache, γίνεται αίτημα για μεταφορά των δεδομένων από τον δίσκο.
- Επιστρέφουμε με typecast σε δείκτη δομής `ext2_inode` τη σωστή διεύθυνση ως `bh->b_data` τη φυσική διεύθυνση του block στη RAM, συν του κατάλληλου byte offset.

```
/* Check the validity of the given inode number. */
if ((ino != EXT2_ROOT_INO && ino < EXT2_FIRST_INO(sb)) ||
    ino > le32_to_cpu(EXT2_SB(sb)->s_es->s_inodes_count))
    goto eival;

/* Figure out in which block is the inode we are looking for and get
 * its group block descriptor. */
block_group = (ino - 1) / inodes_pg;
gdp = ext2_get_group_descriptor(sb, block_group, NULL);

/* Figure out the offset within the block group inode table */
block = le32_to_cpu(gdp->bg_inode_table);
offset = ((ino - 1) % inodes_pg) * inode_sz;
block += offset / blocksize;
offset %= blocksize;

/* Return the pointer to the appropriate ext2_inode */
bh = sb_bread(sb, block);
if (!bh)
    goto eio;

return (struct ext2_inode *) (bh->b_data + offset);
```

ext2_iget

Η συνάρτηση αυτή μετατρέπει το δεδομένο on-disk inode σε in-memory VFS inode. Δέχεται ως παραμέτρους το superblock και το inode number. Εάν έχει οριστεί ήδη, όπως θα καθοριστεί με τη χρήση της `iget_locked()`, επιστρέφει αυτό, αλλιώς εντοπίζει το offset του inode στη μνήμη με τη χρήση της παραπάνω `ext2_get_inode()` και αρχικοποιεί τα πεδία του VFS inode κατάλληλα, διαχωρίζοντας το σε περιπτώσεις, εάν είναι file inode, directory inode ή link inode.

- Με την `i_getlocked()` γίνεται έλεγχος εάν το δεδομένο inode υπάρχει ήδη στην μνήμη καλώντας την `ext2_sops->alloc_inode()`. Εάν επιστρέφει δείκτη που δεν είναι NULL, σημαίνει ότι έχει βρεθεί το inode cache και πιθανώς το κάνει allocate. Το επιστρέφει με το VFS inode lock ώστε να μην μπορεί καμία άλλη διεργασία ή νήμα να το χρησιμοποιήσει. Αφού επιστρέψει τον δείκτη αυτό, τερματίζει.
- Ειδικά, με την `ext2_get_inode()` που υλοποιήσαμε προηγουμένως λαμβάνουμε το raw_inode από τον δίσκο και προχωράμε στην αρχικοποίησή του.
- Ορίζουμε το `i_mode` του inode, που προσδιορίζει το είδος του. Επιπλέον, ορίζουμε τα user και group permissions (uid/gid), το link count, τα timestamps, το νούμερο των blocks allocated για το inode και το μέγεθος του σε bytes.
- Στη συνέχεια, διακρίνουμε τις εξής περιπτώσεις:
 - Με την `S_ISREG`, ελέγχουμε εάν το `i_mode` αντιστοιχεί σε απλό αρχείο, οπότε ορίζουμε τα πεδία `i_op`, `i_for` και `i_mapping` → `a_ops` με τις `ext2_file_inode_operations`, `ext2_file_operations` και `ext2_aops` (address space operations, που λένε στον πυρήνα πώς να διαχειριστεί την page cache), αντίστοιχα.
 - Με την `S_ISDIR`, ελέγχουμε εάν το `i_mode` αντιστοιχεί σε directory, οπότε ορίζουμε τα ίδια πεδία με τις `ext2_dir_inode_operations`, `ext2_dir_operations` και `ext2_aops`, αντίστοιχα.
 - Με την `S_ISLNK`, ελέγχουμε αν το inode αναφέρεται σε symbolic link, δηλαδή αποτελεί αντικείμενο που αναφέρεται στο pathname ενός αρχείου. Εάν είναι fastlink του αναθέτουμε `simple_symlink_inode_operations`, ενώ αν πρόκειται για απλά links αναθέτουμε page-backed operations, τις

page_symlink_inode_operations, που χρησιμοποιούν address-space filesystem operations.

- Ειδικά, πρόκειται για special inode, οπότε του αναθέτουμε ext2_special_inode_operations.
- Επιπλέον, ορίζουμε τα πεδία της δομής ext2_inode_info, η οποία είναι η δομή του inode που βρίσκεται στη μνήμη του πυρήνα. Αρχικοποιούμε τη δομή ei = EXT2_I(inode). Ορίζουμε το dtime (deletion time), τα flags, το block group στο οποίο ανήκει, ενώ αποθηκεύουμε στον πίνακα i_data τα αντίστοιχα δεδομένα του raw inode από τον πίνακα i_block, ο οποίος περιέχει τους δείκτες στα δεδομένα του.
- Με την brelse αποδεσμεύουμε το buffer head για να μπορεί να χρησιμοποιηθεί από άλλες διεργασίες / νήματα.
- Τέλος, με την unlock_new_inode(), το inode μαρκάρεται ως πλήρως αρχικοποιημένο και μπορεί να χρησιμοποιηθεί από άλλη διεργασία/νήμα.

```
//> Setup the {inode,file}_operations structures depending on the type.
if (S_ISREG(inode->i_mode)) {
    inode->i_op = &ext2_file_inode_operations;
    inode->i_fop = &ext2_file_operations;
    inode->i_mapping->a_ops = &ext2_aops;
} else if (S_ISDIR(inode->i_mode)) {
    inode->i_op = &ext2_dir_inode_operations;
    inode->i_fop = &ext2_dir_operations;
    inode->i_mapping->a_ops = &ext2_aops;
} else if (S_ISLNK(inode->i_mode)) {
    if (ext2_inode_is_fast_symlink(inode)) {
        inode->i_op = &simple_symlink_inode_operations;
        inode->i_link = (char *)ei->i_data;
        nd_terminate_link(ei->i_data, inode->i_size, sizeof(ei->i_data) - 1);
    } else {
        inode->i_op = &page_symlink_inode_operations;
        inode_nohighmem(inode);
        inode->i_mapping->a_ops = &ext2_aops;
    }
} else {
    inode->i_op = &ext2_special_inode_operations;
    if (raw_inode->i_block[0])
        init_special_inode(inode, inode->i_mode,
            old_decode_dev(le32_to_cpu(raw_inode->i_block[0])));
    else
        init_special_inode(inode, inode->i_mode,
            new_decode_dev(le32_to_cpu(raw_inode->i_block[1])));
}
```

```

/*
 * Fill the necessary fields of the ext2_inode_info structure.
 */
ei = EXT2_I(inode);
ei->i_dtime = le32_to_cpu(raw_inode->i_dtime);
ei->i_flags = le32_to_cpu(raw_inode->i_flags);
ext2_set_inode_flags(inode);
//ei->i_dtime = 0;
//ei->i_state = 0;
ei->i_block_group = (ino - 1) / EXT2_INODES_PER_GROUP(inode->i_sb);
//> NOTE! The in-memory inode i_data array is in little-endian order
//> even on big-endian machines: we do NOT byteswap the block numbers!
for (n = 0; n < EXT2_N_BLOCKS; n++)
    ei->i_data[n] = raw_inode->i_block[n];

brelse(bh);
unlock_new_inode(inode);
return inode;
}

```

ext2_allocate_in_bg

Υλοποιεί τη δέσμευση blocks από το block bitmap. Δέχεται ως παραμέτρους το superblock, το group number, δείκτη στο bitmap block που μεταβάλλει και τον αριθμό count, ο οποίος είναι ο επιθυμητός αριθμός blocks που οφείλουν να δεσμευτούν. Αναζητά το πρώτο ελεύθερο bit και προσπαθεί να δεσμεύσει το πολύ count συνεχόμενα blocks, χρησιμοποιώντας atomic operations για ασφάλεια. Η χρήση συγχρονισμού είναι απαραίτητη, καθώς θέλουμε πολλαπλά νήματα ή διεργασίες να μπορούν να δεσμεύσουν blocks εντός του ίδιου block group. Για τον λόγο αυτό, θέλουμε να γίνεται συνεχώς προσπάθεια απόκτησης του κλειδώματος.

- Γίνεται εύρεση του πρώτου free bit στο block bitmap με τη χρήση της *find_next_zero_bit_le()*. Εάν αυτό επιστρέφει θέση μεγαλύτερου του τελευταίου bit που θέλουμε να εξετάσει, επιστρέφουμε -1 και δεν γίνεται δέσμευση.
- Στη συνέχεια, γίνεται προσπάθεια με τη χρήση της *ext2_set_bit_atomic()* να δεσμευτεί το block που βρέθηκε διαθέσιμο. Εάν κάποιο νήμα πρόλαβε πρώτο να το δεσμεύσει, επιστρέφει μη μηδενικό αριθμό, οπότε γίνεται increment του *first_free_bit*, ώστε να χρησιμοποιηθεί ως offset για να γίνει αναζήτηση νέου bit όταν επαναλαμβάνεται η *find_next_zero_bit_le()*.

- Αφού βρεθεί ένα διαθέσιμο bit, γίνεται προσπάθεια να δεσμεύσουμε count συνεχόμενα blocks, υπό τη συνθήκη ότι δεν ξεπερνάμε τον μέγιστο αριθμό blocks.
- Εάν κάποια προσπάθεια δέσμευσης block αποτύχει, θα τερματίσει ο βρόχος. Η count θα λάβει την τιμή του num, ώστε να γνωρίζουμε πόσα blocks δεσμεύτηκαν τελικά.
- Τέλος, επιστρέφουμε το first_free_bit ως το offset μέσα στο block bitmap του πρώτου block που δεσμεύσαμε.

```

struct ext2_sb_info *sbi = EXT2_SB(sb);

repeat:
    first_free_bit = find_next_zero_bit_le(bitmap_bh->b_data, nblocks, first_free_bit);

    if (first_free_bit >= nblocks)
        return -1;

    // try to claim the first free bit
    if (ext2_set_bit_atomic(sb_bgl_lock(sbi, group), first_free_bit, bitmap_bh->b_data)) {
        first_free_bit++;
        goto repeat;
    }

    // try to allocate consecutive blocks
    num = 1;
    while ((num < *count) && (first_free_bit + num < nblocks)) {
        if (ext2_set_bit_atomic(sb_bgl_lock(sbi, group),
                                first_free_bit + num, bitmap_bh->b_data))
            break;
        num++;
    }

    *count = num;
    return first_free_bit;

```


Με τον παραπάνω κώδικα εξασφαλίσαμε σωστή χαρτογράφηση disk inode → VFS inode, σωστό layering μεταξύ VFS και ext2-private δομών και σταθερή λειτουργία του filesystem.

Η επιτυχής εμφάνιση του καταλόγου lost+found μετά το mount αποδεικνύει ότι:

- το superblock διαβάστηκε σωστά
- το root inode (2) διαβάστηκε σωστά
- η ανάγνωση directory entries λειτουργεί κανονικά

```
root@utopia:/home/user/shared/ext2/ext2-lite-helpcode-20251203-0# cat /proc/mounts | grep "ext2lite"
/dev/loop0 /mnt/ext2lite ext2-lite rw,relatime,errors=continue 0 0
```

```
root@utopia:/home/user/shared/ext2/ext2-lite-helpcode-20251203-0# insmod ./ext2-lite.ko
root@utopia:/home/user/shared/ext2/ext2-lite-helpcode-20251203-0# lsmod
Module                Size  Used by
ext2_lite              53248   0
root@utopia:/home/user/shared/ext2/ext2-lite-helpcode-20251203-0# mount -t ext2-lite -o loop ./ext2-lite.img /mnt/ext2lite
root@utopia:/home/user/shared/ext2/ext2-lite-helpcode-20251203-0# ls -la /mnt/ext2lite
total 19
drwxr-xr-x 4 root root 1024 Jan  4 19:33 .
drwxr-xr-x 4 root root 4096 Jan  4 18:51 ..
-rw-r--r-- 1 root root   11 Jan  4 19:33 hello.txt
drwx----- 2 root root 12288 Jan  4 19:12 lost+found
drwxr-xr-x 2 root root 1024 Jan  4 19:33 testdir
```

```
root@utopia:/home/user/shared/ext2/ext2-lite-helpcode-20251203-0# cat /mnt/ext2lite/hello.txt
hello ext2
```